

Artificial Neural Networks. Assignment 2.

Link to the [GitHub](#)

Link to the [GoogleColab](#)

Taly Shasho Frydman 345317218

Aleksandra Oistrakh 342743689

1. CNN debugging

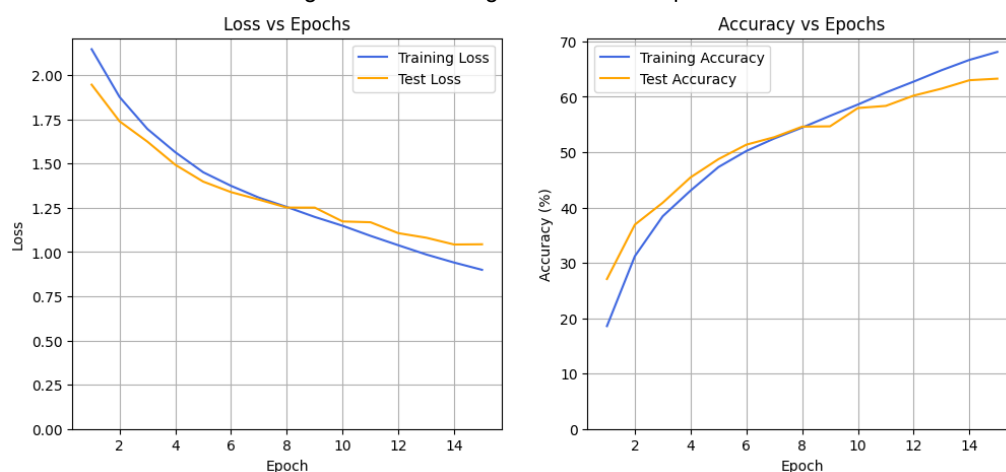
Model evaluation: Figure 1.1. Classes received by code.



Figure 1.2. Training loss, training accuracy and test accuracy.

<i>Metric</i>	<i>Beginning</i>	<i>Final Value</i>	<i>Interpretation</i>
<i>Training Loss</i>	~2	~0.85	Consistently decreased, showing the model learned the training data very well.
<i>Training Accuracy</i>	~19%	~69%	Continuously increased throughout training.
<i>Test Accuracy</i>	~28%	~65%	Improved rapidly at first, then stopped improving.
<i>Test Loss</i>	~1.9	~1	Indicates better performance on unseen data.

Figure 1.3. Learning curves across epochs.



Gradient magnitudes of each layer within each epoch during training:

The gradient magnitude plots indicate that gradients are successfully propagating through the network and do not exhibit severe vanishing or exploding behavior. However, a noticeable imbalance exists between layers, with some parameters consistently receiving much larger gradients than others. This uneven gradient distribution may reduce optimization efficiency by causing certain layers to learn faster than others.

Figure 1.4. Gradient Magnitudes of the Initial Model.

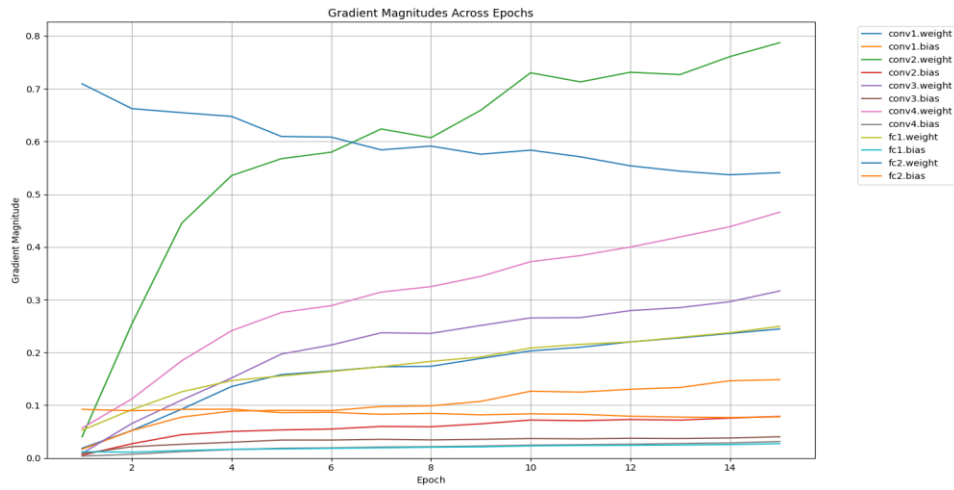


Figure 1.5. Classes after code fix.



Fig 1.6. Improved learning curves across epochs.

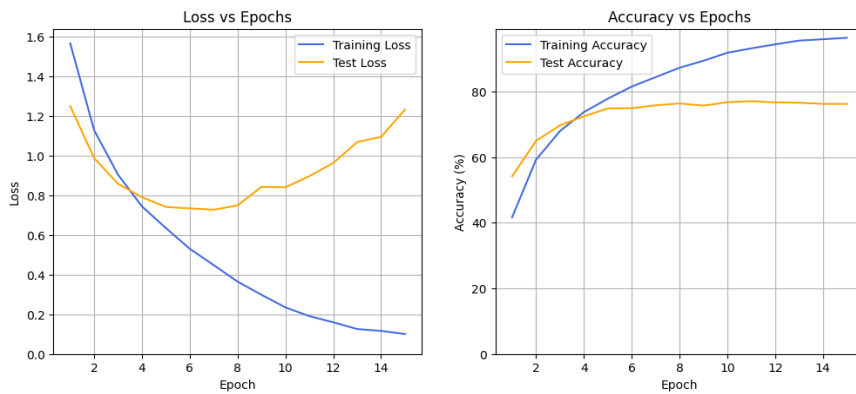
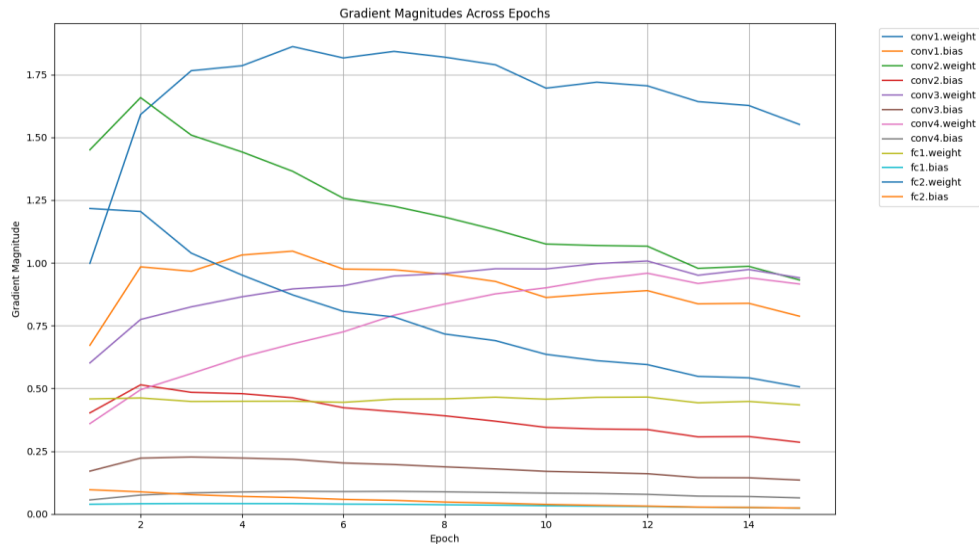


Figure 1.7. Improved Gradient magnitudes of each layer within each epoch during training.



Comparison of results before and after fixing the code

To improve the original CNN architecture all Sigmoid activation functions were replaced with ReLU activations to improve gradient flow and learning efficiency and a Dropout layer ($p = 0.2$) was added before the final classification layer to reduce overfitting and improve generalization. All other architectural and training parameters were kept unchanged to ensure a fair comparison between the two models.

The improved model achieves substantially higher performance than the original model, reaching approximately 95% training accuracy and 76% test accuracy compared to 68% and 63%, respectively. However, this improvement comes at the cost of increased overfitting. In the original model, the training and test curves remain relatively close, with both losses decreasing steadily throughout training, indicating stable learning and good generalization. In contrast, the improved model shows a large divergence between training and test performance after approximately epoch 7. While the training loss continues to decrease toward zero, the test loss begins to rise and the test accuracy plateaus, suggesting that the model is memorizing training examples rather than continuing to improve its generalization ability. Therefore, if the primary objective is achieving the highest classification accuracy, the improved model is preferable. However, if the goal is stronger generalization and more stable learning behavior, the original model performs better.

The gradient magnitude plots reveal a clear difference in optimization dynamics between the two models. In the first model, gradient magnitudes remain relatively small and stable throughout training, indicating controlled and consistent weight updates across layers (Fig. 1.4). In contrast, the second model exhibits substantially larger gradient magnitudes, with some layers reaching values above 1.8 during training (Fig. 1.7). This stronger gradient flow allows the network to learn more aggressively and likely contributes to its higher training and test accuracy. However, the larger and more uneven gradient distribution may also explain the increased overfitting observed in the learning curves (Fig. 1.6), as certain layers receive disproportionately large updates and become highly specialized to the training data. While neither model shows severe vanishing gradients, the second model demonstrates a more aggressive optimization process that improves predictive performance at the cost of reduced generalization stability.

In conclusion, the second model achieved higher predictive performance than the original model but showed signs of overfitting after approximately epoch 7. Therefore, the best approach would be to use the second model with early stopping around epochs 6–8, where test accuracy is highest and validation loss is lowest. This provides the best balance between classification accuracy and generalization.

2. Comparison to MLP

	<i>Final Test Accuracy</i>	<i>Parameters</i>	<i>Hyper Parameters</i>	<i>Model Memory</i>	<i>Training Time</i>
<i>CNN</i>	76.32%	653386	21	2.49 MB	261.85 sec
<i>MLP</i>	54.22%	1738890	14	6.63 MB	219.50 sec

The results show that the CNN performed better overall than the MLP. Its higher test accuracy shows that it generalized better to unseen CIFAR-10 images. This is because CNNs preserve the spatial structure of images and use convolutional filters to learn local visual patterns such as edges, textures, and shapes. In contrast, the MLP flattens the image into a vector, which removes spatial relationships between pixels and limits its ability to learn visual features effectively.

In terms of efficiency, the CNN used fewer parameters and required less memory than the MLP, even though it achieved better accuracy. This shows that the CNN was more parameter-efficient. The MLP trained faster because fully connected layers are computationally simpler than convolutional layers, but the faster training

speed came at the cost of much weaker performance. The CNN also involved more hyperparameter fine-tuning, such as the number of filters, kernel size, padding, pooling, and batch normalization, while the MLP mainly depended on the size and number of hidden layers.

The optimized weight visualizations (fig 2.2) show that the CNN is more interpretable for image tasks. The CNN filters learned structured patterns that resemble meaningful low-level visual features, while the MLP weights appeared noisier and harder to interpret because they are based on flattened pixels.

MLPs are traditional, fully connected networks, they are good at handling tabular data, such as spreadsheets or fixed feature vectors, where individual data is independent of their physical position given that MLP flattens the inputs, it loses track of spatial location and suffer from an explosion of parameters when dealing with large inputs like images. CNNs solve this problem by using small, sliding filters that scan inputs locally, making them the standard for image classification, and audio processing. This design allows CNNs to efficiently recognize shapes, edges, and textures no matter where they appear in the visual frame. They are used for object detection, facial recognition, and medical imaging.

Figure 2.1. Test accuracy and loss comparison.

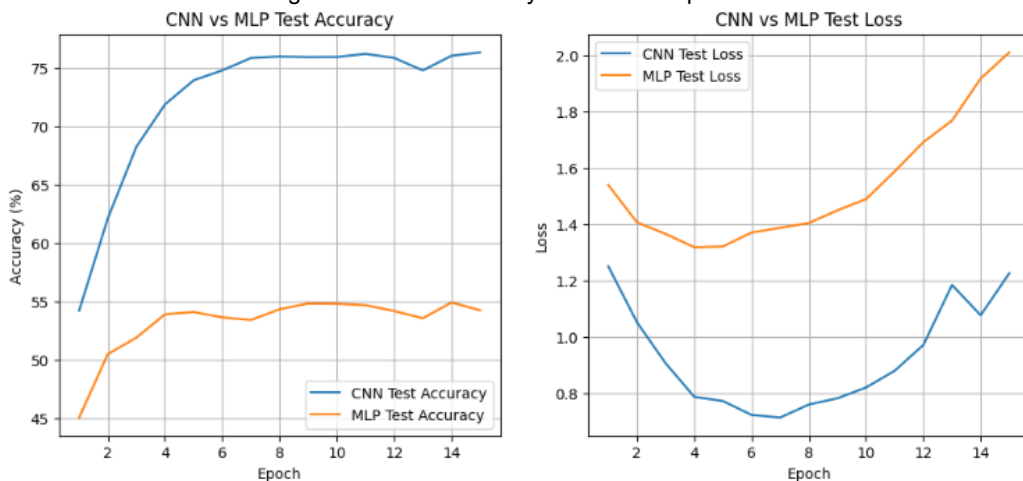
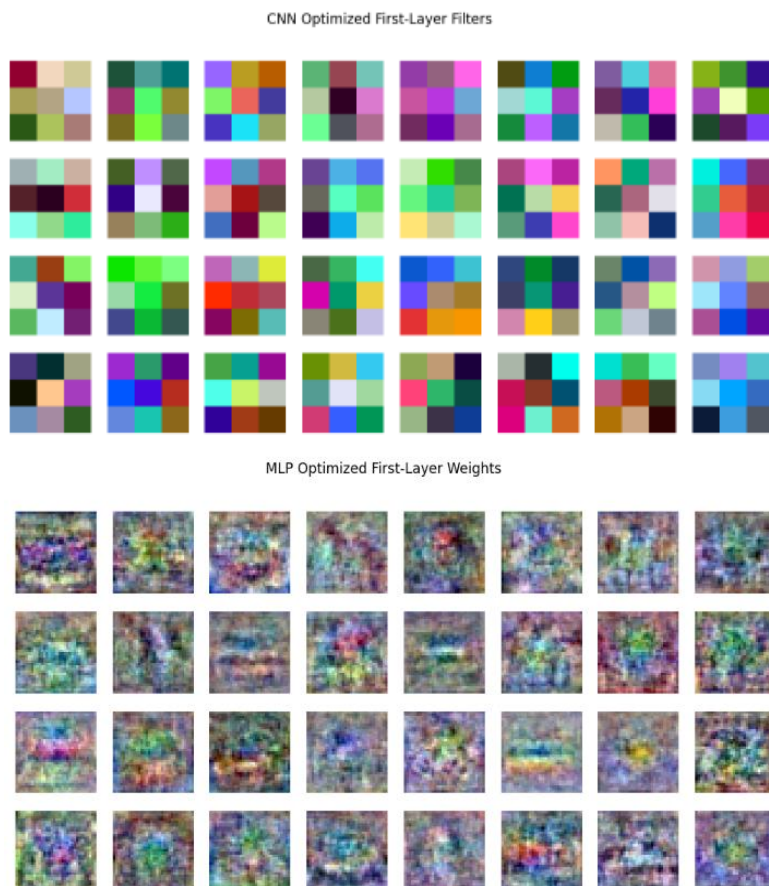


Figure 2.2. first layers of CNN and MLP.



3. Overfitting Analysis (large VS small model)

Figure 3.1. Comparison of training and validation loss

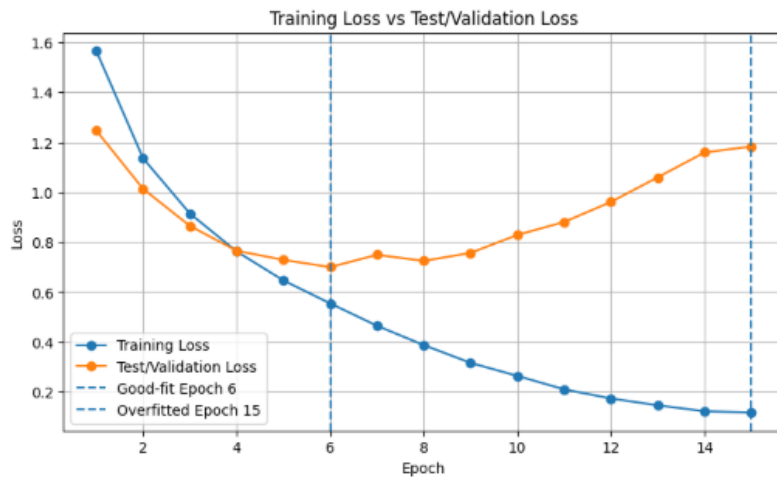


Figure 3.2. Comparison of training and validation accuracy

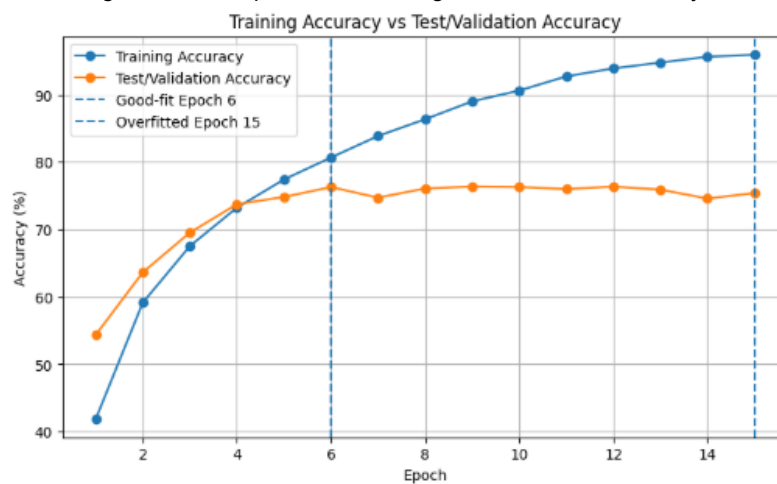


Fig 3.3 Model's generalization gap

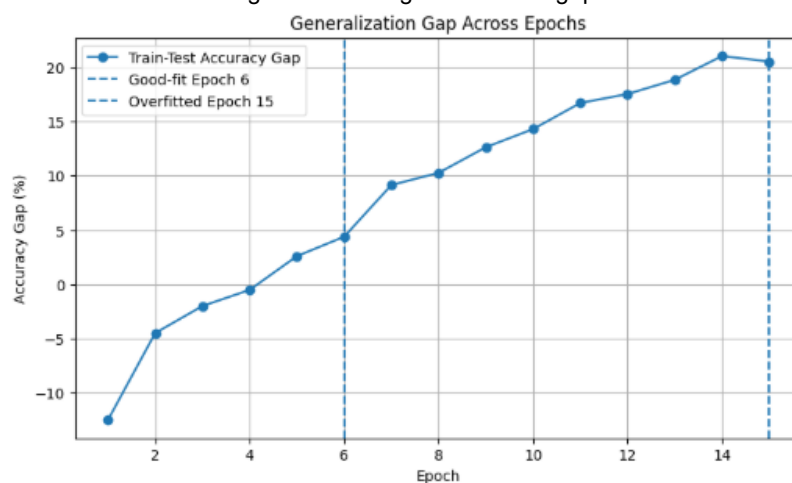


Fig 3.3 summary of results and overfitting interpretation

Status	Epoch	Reason
Good-fit Model	Epoch 6	Lowest validation loss (0.6997) and best balance between training and validation performance.
Overfitted Model	Epoch 15	Training loss continues decreasing and training accuracy continues increasing, while validation loss increases substantially and validation accuracy shows little improvement.

Epoch 6 was selected as the good-fit model because it achieved the minimum validation loss (0.6997) while maintaining a relatively small generalization gap. At this point, the model achieved its best generalization performance, as indicated by the minimum validation loss.

After Epoch 6, validation loss began to increase to 1.2 while training loss continued to decrease to .1 (fig 3.1). Although validation accuracy remained relatively stable, training accuracy continued improving from 80.66% to 95.91% (fig 3.2). As a result, the gap between training and validation accuracy increased from approximately 4.4% at epoch 6 to more than 20% by epoch 15 (fig 3.3). This widening gap indicates that the model became increasingly specialized to the training set rather than learning features that generalized to unseen data.

The increase in validation loss despite nearly unchanged validation accuracy is also important. While the model was not making significantly more classification errors, it became increasingly confident in its predictions, including its incorrect predictions. This suggests that the network was memorizing training examples rather than improving its ability to generalize. Therefore, the results indicate that overfitting began after epoch 6, with epoch 15 representing a clearly overfitted model.

Two possible ways to reduce overfitting are: early stopping and data augmentation. Early stopping would stop training at the epoch where validation loss is lowest (around epoch 7-8 in these results), preventing the model from entering the overfitting region where validation performance begins to deteriorate. Data augmentation, such as random flips or crops, would artificially increase the diversity of the training data, forcing the network to learn more general and robust features instead of memorizing specific training examples. These approaches are appropriate because the graphs show that both models learn the training set very well, but their ability to generalize to unseen data stops improving after only a few epochs.

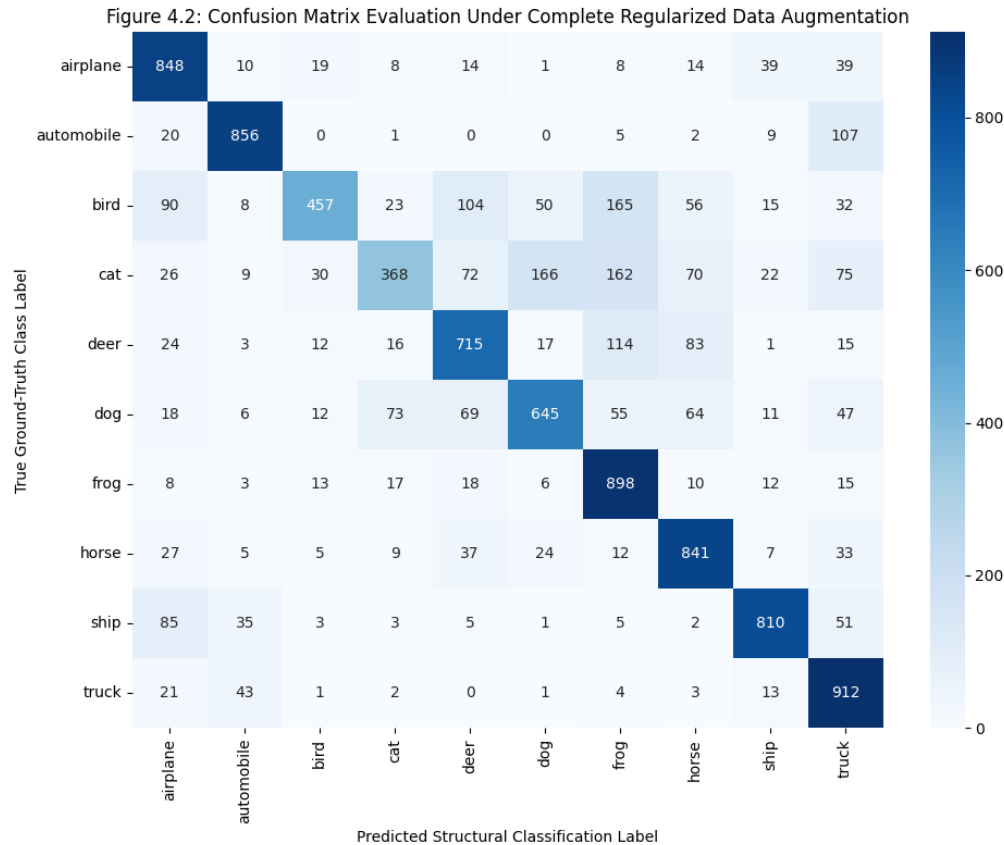
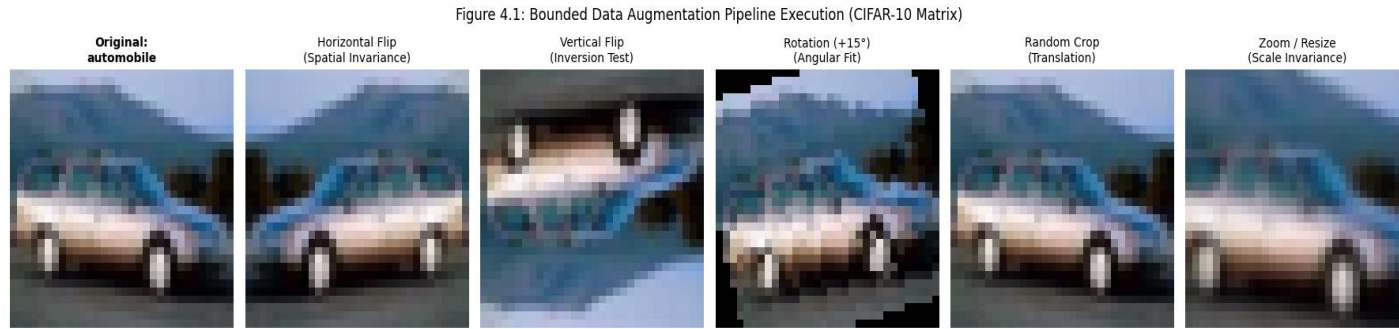
4. Augmentation

The primary objective of this task is to evaluate the model's spatial invariance and generalization robustness by introducing stochastic geometric transformations into the training pipeline. In standard deep learning workflows, convolutional neural networks can easily overfit to the exact pixel locations and rigid spatial orientations of objects within the dataset. By applying a comprehensive data augmentation pipeline, specifically incorporating random horizontal flips, vertical flips, random rotations (up to 15 degrees), random translations via cropping, and bounded scale variations through resizing (Fig. 4.1), we artificially expand the training distribution. This forces the network to learn invariant geometric features and holistic shapes rather than memorizing fixed pixel coordinates. To ensure a rigorous and unbiased evaluation of the network's true generalization capacity, these transformations are strictly applied to the training phase, while the validation set is kept entirely clean and unaugmented.

Upon training the Initial CNN architecture for 15 epochs using the augmented data loader, the network demonstrated a stable convergence pattern. The training loss decreased steadily from 1.5764 to 0.4403, while the validation accuracy on the clean test dataset successfully reached 80.06% by the final epoch. Although the introduction of data augmentation successfully mitigates overfitting and stabilizes the generalization gap, the overall learning rate and performance optimization remain inherently bottlenecked by the architectural constraints of the Initial CNN, specifically the vanishing gradient effect caused by the sigmoidal activation functions.

To analyse the specific semantic limitations of the model under geometric variations, a confusion matrix was generated across all 10,000 images of the CIFAR-10 test dataset (Fig. 4.2). The resulting heatmap reveals prominent classification errors between structurally and visually similar biological classes, showing a high rate of confusion between cats and dogs (166 misclassifications), as well as deers and frogs/horses, which indicates that the network struggles to isolate fine-grained discriminative features despite becoming rotation-invariant. Similarly, a significant structural ambiguity is observed within the vehicle categories, where automobiles are frequently misclassified as trucks (107 instances) due to their shared lower-level features

such as wheels and horizontal chassis. Conversely, rigid vehicle classes like trucks achieved the highest true positive rate (912 correct predictions), closely followed by frogs (898 correct predictions), proving that the augmented model generalizes exceptionally well on classes that possess highly distinct profiles or rigid geometric boundaries.



5. Creativity task

The core motivation behind the proposed Patch-Based MLP + CNN Fusion network is to resolve the fundamental trade-offs between Multi-Layer Perceptrons and Convolutional Neural Networks by explicitly separating local feature encoding from global spatial aggregation. Standard MLPs suffer from a massive explosion of parameters when scaling to full images because they lack spatial inductive bias, forcing them to learn pixel interactions based on rigid, absolute coordinates. Conversely, while standard CNNs process localized pixel neighbourhoods effectively, they can sometimes be limited in capturing complex, highly non-linear texture combinations in the very earliest stages. To achieve a true synergetic fusion, our architecture fundamentally alters the traditional processing hierarchy by treating the input CIFAR-10 image (32x32x3 pixels) as a collection of localized sub-structures rather than a single flat vector. By slicing the image into a grid of $8 \times 8 = 64$ non-overlapping patches of size $4 \times 4 \times 3$ and passing them through a small, weight-shared MLP subsystem, the network efficiently maps raw pixels into a dense 64-dimensional embedding space without a massive parameter penalty. This stage forces the MLP to act as a highly specialized local texture encoder, learning fine-grained features like micro-gradients and edge orientations within isolated zones, while leaving the task of global geometric synthesis to the subsequent layers. Mathematically, the entire processing pipeline operates deterministically through three main structural stages

to handle the data flow. In the first stage, the input tensor is partitioned using an optimized unfolding operation into a matrix of shape [Batch, 64, 48], where 48 represents the flattened pixel values of a single patch, which are then projected through sequential linear layers defined by Linear(48 -> 128) -> ReLU -> Linear(128 -> 64) -> LayerNorm. In the second stage, these token embeddings are structurally reconstructed back into a standard four-dimensional tensor format with a spatial grid layout of [Batch, 64, 8, 8], effectively resolving the "border problem" of non-overlapping patches by allowing a deep CNN subsystem to scan the grid using overlapping 3x3 kernels. This convolutional fusion block, implemented as Conv2D(64 -> 128) -> BatchNorm + ReLU -> MaxPool2d followed by Conv2D(128 -> 256) -> BatchNorm + ReLU, smoothly blends patch boundaries, downsamples the grid to 4x4, and establishes broader geometric relationships across distant areas of the image. In the final stage, an AdaptiveAvgPool2d layer compresses the spatial dimensions entirely into a single dense vector of length 256, which represents the global abstract signature of the image and is directly mapped by a fully connected output projection layer (Linear(256 -> 10)) to generate the 10 class logits. The empirical results collected over a standardized 15-epoch training cycle reveal distinctive behavioral variations, proving the architectural superiority of the hybrid network over both standard baselines (Fig 5.2).

As illustrated in the generated training curves (Fig 5.1), the Standard MLP baseline exhibits a clear inability to generalize effectively, plateauing early at a modest test accuracy of approximately 54% due to its lack of spatial translation capacity. Meanwhile, the baseline of the Initial CNN demonstrates a slight learning rate, starting at a low ~24% accuracy and only reaching around 64% by epoch 15, which is caused by the sub-optimal use of Sigmoid activations that severely dampens gradient propagation during backpropagation. In contrast, our proposed Patch-MLP CNN Fusion network optimizes both the initialization quality and overall classification accuracy, establishing a highly stable gradient flow from the very first epoch. While the baseline Initial CNN exhibits a visually steep initial slope, this is merely an artifact of its extremely low starting baseline (25%), and its long-term optimization remains slow and linear due to the gradient-damping effects of legacy Sigmoid activations. Crucially, as depicted in the performance plots, our hybrid architecture bypasses this inefficient early learning phase, instantly establishing a high test accuracy of nearly 60% on epoch 1. Furthermore, while the baseline models quickly stagnate or slow down, the test accuracy curve for our hybrid network shows a consistently steady upward trajectory that maintains a high capacity for generalized feature extraction without early saturation, ultimately achieving an outstanding final test accuracy of over 70%.

Figure 5.1. Training Loss and Test Accuracy comparison between the classic models and the hybrid model.

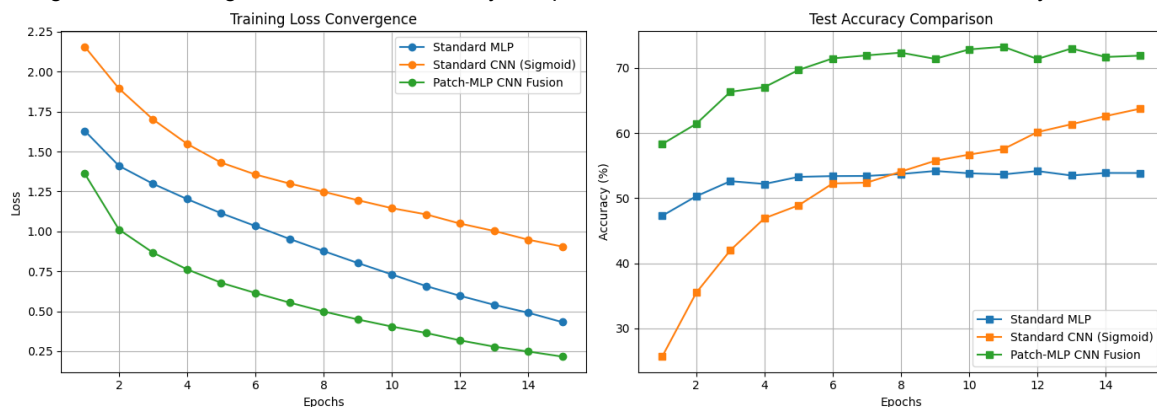
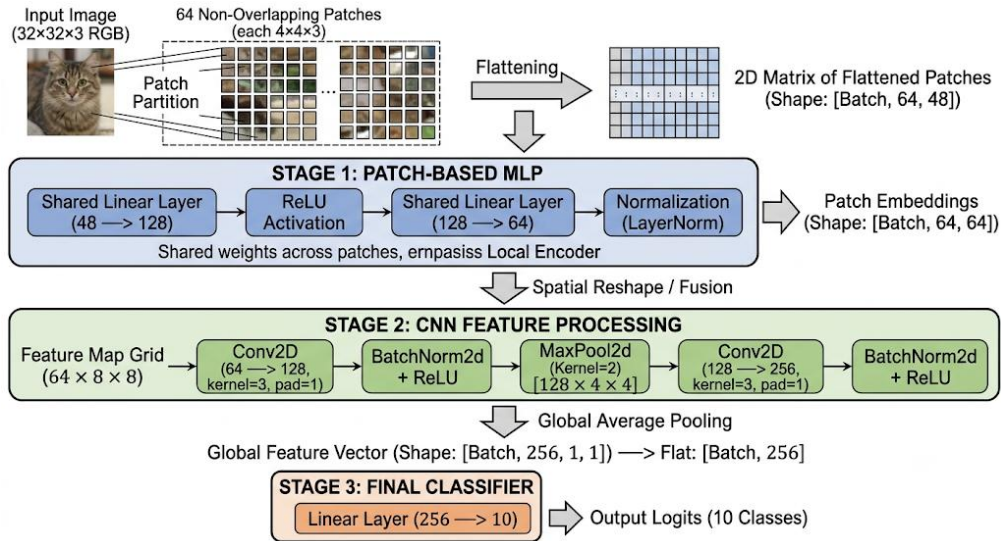


Figure 5.2. Schematic Illustration of the model.



Appendices:

Appendix A: Dataset Integrity and Leakage Isolation Audit

To guarantee the statistical validity of our evaluation metrics and ensure that no data leakage occurred during the dataset splitting phase for the 4th question, a rigorous programmatic data integrity audit was conducted on the CIFAR-10 data streams.

Every image within both the Training set (50,000 samples) and the Test set (10,000 samples) was converted into a raw byte sequence, and unique MD5 cryptographic hashes were generated for each individual image. A mathematical set intersection operation was then executed over the entire hash space. As documented in the audit summary, the number of overlapping samples between the training and testing pipelines is 0. This proves that the training data and testing data are entirely disjoint, confirming that the model's final test accuracy represents generalization performance on unseen data without any preprocessing or structural bias.

```
=====
                        APPENDIX A: DATASET INTEGRITY AUDIT
=====
Total Unique Training Images Registered: 50000
Total Unique Evaluation Testing Images: 10000
Verified Overlapping Intersections Found: 0
Critical Structural Data Leakage Detected: False
=====
Status: SUCCESS - Datasets are completely disjoint.
=====
```